

CANDY

DIALOGUE ENGINE

Choice UI Customization - Basic

1.1.0

Before proceeding further, make sure you read the [Player Choices](#) guide and that you have a comfortable understanding of the §Choice_List command and the various pieces of data it §Choice_List lines contain. Understanding the §Choice_Status and §Timer_Status commands is also recommended.

Choice UI Elements are used by the §Choice_List command, which displays a list of choices to the player. A choice UI consists of three elements:

- 1) A menu scene and script.
- 2) Category scenes and scripts.
- 3) Choice scenes and scripts.

Menu Scenes

Menu scenes act as the main choice UI wrapper. They contain category scenes as children, and display one category at a time.

Nodes

Menu scenes require only three nodes: a root node, a node that acts as a parent for category scenes, and one that acts as a parent for timers.

You can give menu scenes a visual appearance, such as a PanelContainer or TextureRect, and you can display text such as a menu title or a prompt. However, this is completely unnecessary: the menu scene can be made entirely of non-visible elements if you wish.

That being said, it should be able to fully display its child category scenes on screen (i.e. ensure it's not too small or that it doesn't otherwise conceal or obstruct categories).

You may also want to display timers visually, such as a numerical countdown or a graphic like a progress bar. You'll need to decide how you want to display timers in the UI, and possibly which ones/how many (if there are multiple). Note that displaying timers requires writing your own code in the menu scene's script.

Root

The root node can be named freely.

Required Nodes

Root Node

- Holds Category List and Timer List nodes as children (can be indirect children, i.e. grandchildren).
- Can be any node type.
- No script pointer required.

Category List

- Category scenes will automatically be made children of it.
- No other node should be a children of it.
- Can be any node type.
- Script pointer: `@export var category_list`

Timer List

- Choice timers will automatically be made children of it.
- No other node should be a children of it.
- Can be any node type.
- Script pointer: `@export var timer_list`

Optional Nodes

Background/Frame

- Suggested: TextureRect, PanelContainer, etc.
- Give a visual appearance to the menu.

Title

- Suggested: Label or RichTextLabel
- For displaying the menu's title to the player.
- Script pointer: `@export var title_node`

Prompt

- Suggested: Label or RichTextLabel
- For displaying a prompt to the player.
- Script pointer: @export var **prompt_node**

Timer Displays

- Text or visual nodes (e.g. ProgressBar, ColorRect, TextureRect, Sprite2D...)
- For displaying timers, either as a numeric countdown or a graphical illustration such as a progress bar.
- Script pointer: @export var **timer_list**

Shield

- Suggested: ColorRect, PanelContainer
- For covering the Choice Menu and prevent clicking when choices are selected or timers have timed out.
- Script pointer: @export var **shield**

Script

Choice menus can be assigned the **Choice_Menu.gd** script to their root node. The script can be customized to fit your needs.

Variables

The following variables are required in the script:

caller

Stores a reference to the dialogue engine instance that created the choice menu.

nesting_depth

Stores the nesting depth the choice menu was created at.

title

Stores the general "Title" value of the §Choice_List command. Used optionally for displaying a menu title.

prompt

Stores the general "Prompt" value of the §Choice_List command. Used optionally for displaying a menu prompt.

custom

Stores the general "Custom" value of the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

tags

Stores the general "Tags" value of the §Choice_List command. Used by commands that modify opened choice menus to target the right menus.

Functions**Required:****initialization_finished()**

Called by the §Choice_List command once the menu, categories and choice buttons have been setup.

The function has no code by default: you can add custom logic that you want it to run.

_on_timer_timeout()

Called when a choice timer expires, and emits the choice_list_event() signal to notify the §Choice_List command.

deactivate()

Called by the §Choice_List command when an event makes the command perform a Bridge transition. The basic purpose of this function is to hide the choice menu until the dialogue engine returns from the Bridge.

You can add custom logic to the function if you want.

reactivate()

Called by the §Choice_List command upon returning from a Bridge transition. The basic purpose of this function is to unhide the choice menu so the player can continue using it.

The function can also be called manually (e.g. with a §Call command inside the Bridge) to reactivate the menu sooner.

You can add custom logic to the function if you want.

_set_general_title()

Displays the title of the menu, if you have a node for that purpose. Note that you must edit the function to provide the path to that node in your menu scene.

Notice that the default code uses of localization dictionaries (Candy_Localizations.gd) to automatically display the text in the selected language.

_set_general_prompt()

Displays a prompt for the menu, if you have a node for that purpose. Note that you must edit the function to provide the path to that node in your menu scene.

Notice that the default code uses of localization dictionaries (Candy_Localizations.gd) to automatically display the text in the selected language.

Optional:

_ready()

Calls the [_set_general_title\(\)](#) and [_set_general_prompt\(\)](#) functions. Doing so is optional and can be removed if you don't want to display a title and/or a prompt in the menu.

x_initialization_finished()

This is a hook function, called by the §Choice_List command once it has finished initializing the Choice Menu, Categories and Choices.

Specifically, it's called after the Menu, Categories, Choice Buttons have been instantiated, Timers have started, and after the Setup transition of all these elements has been run. It's the last step before the command begins waiting for a choice to be selected.

If necessary, you can move it higher in the command's logic, or insert extra hooks of your own.

hide_old_menus()

This is a convenience function, not used anywhere by default but which you may find helpful for custom purposes. It iterates through all opened choice menus and sets visible = false on all of them,

except for a) the menu instance the function was called on and b) menu instances that have `'never_hide = true'`

Note that it hides menus by calling `deactivate()` on them. If you modify the `deactivate()` code, `hide_old_menus()` will trigger the effects of that code. You can modify `hide_old_menus()` to set `'visible = false'` directly, instead of calling `deactivate()`.

Category Scenes

Category scenes display a choice scene for each choice in the category.

Nodes

Category scenes require only one nodes: a node that acts as a parent for choice button scenes.

Like menu scenes, you can give category scenes a visual appearance, such as a `PanelContainer` or `TextureRect`, and you can display text such as a menu title or a prompt. However, this is completely unnecessary: the menu scene can be made entirely of non-visible elements if you wish.

That being said, a category scene should be able to fully display its child choice button scenes on screen (i.e. ensure it's not too small or that it doesn't otherwise conceal or obstruct categories; you may want to use a `VBoxContainer` to list buttons vertically, possibly as a child of a `ScrollContainer`).

Root

The root node can be named freely.

It will be renamed to match the category it represents when instantiated by the `$Choice_List` command.

Required Nodes

Choice Button List

- Choice button scenes will automatically be made children of it.
- No other node should be a children of it.
- Can be any node type.
- Script pointer: `@export var button_list`

Optional Nodes

Background/Frame

- Suggested: `TextureRect`, `PanelContainer`, etc.
- Give a visual appearance to the menu.

Title

- Suggested: Label or RichTextLabel
- For displaying the category's title to the player.
- Script pointer: @export var **title_node**

Prompt

- Suggested: Label or RichTextLabel
- For displaying a category prompt to the player.
- Script pointer: @export var **prompt_node**

Script

Choice Categories can be assigned the **Choice_Category.gd** script to their root node. The script can be customized to fit your needs.

Variables

The following variables are required in the script:

caller

Stores a reference to the dialogue engine instance that created the choice menu.

parent_menu

Stores a reference to the menu scene that the category is a child of.

general_title

Stores the general "Title" value of the \$Choice_List command.

category_title

Stores the category's "Title" value in the \$Choice_List command. Used optionally for displaying a category title.

general_prompt

Stores the general "Prompt" value of the \$Choice_List command.

category_prompt

Stores the category's "Prompt" value in the §Choice_List command. Used optionally for displaying a category prompt.

general_custom

Stores the general "Custom" value of the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

category_custom

Stores the category's "Custom" value in the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

tags

Stores the category's "Tags" value in the §Choice_List command. Used by commands that modify opened choice menus to target the right categories.

Functions

Required:

_set_category_title()

Displays the title of the category, if you have a node for that purpose. Note that you must edit the function to provide the path to that node in your menu scene.

Notice that the default code uses of localization dictionaries (Candy_Localizations.gd) to automatically display the text in the selected language.

_set_category_prompt()

Displays a prompt for the category, if you have a node for that purpose. Note that you must edit the function to provide the path to that node in your menu scene.

Notice that the default code uses of localization dictionaries (Candy_Localizations.gd) to automatically display the text in the selected language.

Optional:

_ready()

Calls the `_set_category_title()` and `_set_category_prompt()` functions. Doing so is optional and can be removed if you don't want to display a title and/or a prompt in the menu.

Choice Button Scenes

Choice scenes typically consist of a button, which players would click to select a given choice, although it's theoretically possible to use different designs.

Nodes

Choice Button scenes

Root

The root node can be named freely.

It will be renamed to match the choice it represents when instantiated by the `$Choice_List` command.

Required Nodes

Button or TextureButton

- Lets the player select the choice it represents.
- Connect the node's `pressed()` signal to the `_choice_selected()` function in the choice button script.
- Script pointer: `@export var choice_button`

Choice Label

- A node that displays the label text for the choice. Can be the Button/TextureButton itself, or a separate Label or RichTextLabel node.
- Script pointer: `@export var choice_label`

Optional Nodes

Choice Tooltip

- A node that displays the tooltip text for the choice when hovered. Can be the Button/TextureButton itself, or a separate Label or RichTextLabel node.
- Script pointer: `@export var choice_tooltip`

Script

Choice buttons can be assigned the **Choice_Button.gd** script to their root node. The script can be customized to fit your needs.

Variables

The following variables are required in the script:

caller

Stores a reference to the dialogue engine instance that created the choice menu.

parent_category

Stores a reference to the category scene that the choice is a child of.

parent_menu

Stores a reference to the menu scene that the parent category is a child of.

choice_name

Stores the name of the choice in the §Choice_List command.

category_name

Stores the name of the choice's parent category in the §Choice_List command.

general_custom

Stores the general "Custom" value of the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

category_custom

Stores the parent category's "Custom" value in the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

choice_custom

Stores the choice's "Custom" value in the §Choice_List command. Used optionally for configuring custom code in the script, similarly to a function's arguments.

tags

Stores the choice's "Tags" value in the §Choice_List command. Used by commands that modify opened choice menus to target the right categories.

label

Stores the choice's "Label" value in the §Choice_List command. Used for displaying a label text.

tooltip

Stores the choice's "Tooltip" value in the §Choice_List command. Used for displaying a tooltip text.

choice_enabled

Tracks whether the choice is enabled. Theoretically redundant as disabled choices don't have a button scene.

choice_active

Tracks whether the choice is active or inactive. Inactive choices have a button, but can't be selected (no effect when clicked).

choice_invisible

Tracks whether the choice is visible or invisible. Invisible choices have their root node's visible property set to 'false'.

Functions**Required:****setup()**

Called by the §Choice_List command.

Calls several of the functions below, to configure the choice button scene.

_choice_selected()

Called when the choice button is pressed, or if a timer selects the choice.

Emits the choice_list_event signal in the 'caller' dialogue engine instance. When the signal is emitted, the §Choice_List command runs the "Selected" transition assigned to the choice.

The code lines in the function should not be modified unless you know what you're doing, but custom logic can be added.

`_toggle_enabled()`

Called by `_ready()` and by the `$Choice_Status` command. Triggers various effects depending on the value of `choice_enabled`.

No effects are programmed by default: you must write custom code if you desire any effects.

`_toggle_active()`

Called by `_ready()` and by the `$Choice_Status` command. Triggers various effects depending on the value of `choice_active`.

No effects are programmed by default: you must write custom code if you desire any effects.

`_toggle_invisible()`

Called by `_ready()` and by the `$Choice_Status` command. Triggers various effects depending on the value of `choice_invisible`.

By default, toggles the 'visible' property of the choice button scene. You can write custom code if you desire additional effects.

`_set_label()`

Displays the label text in the `choice_label` node.

Notice that the default code uses of localization dictionaries (`Candy_Localizations.gd`) to automatically display the text in the selected language.

`_set_tooltip()`

Assigns the tooltip text to the `choice_tooltip` node. The code can be customized to fit your choice button scene.

Notice that the default code uses of localization dictionaries (`Candy_Localizations.gd`) to automatically display the text in the selected language.